

Stripe Setup Guide

Complete guide for setting up Stripe payment processing in your E-Commerce Automotive Store.

Table of Contents

1. [Creating a Stripe Account](#)
 2. [Getting API Keys](#)
 3. [Webhook Configuration](#)
 4. [Testing Payments](#)
 5. [Going Live Checklist](#)
 6. [Currency Configuration](#)
 7. [Common Issues & Solutions](#)
 8. [Payment Flow Overview](#)
-

Creating a Stripe Account

Step 1: Sign Up

1. Go to <https://stripe.com>
2. Click “Start now” or “Sign up”
3. Enter your email address
4. Create a strong password
5. Verify your email

Step 2: Complete Business Profile

After signing up, complete your business details:

1. **Business Type:**
 - Individual
 - Company
 - Non-profit
2. **Business Information:**
 - Legal business name
 - Business address
 - Phone number
 - Website URL
 - Business description: “Automotive parts and accessories retailer”
3. **Industry:**
 - Select: “Automotive” or “Retail”
4. **Personal Information:**
 - Legal name

- Date of birth
- Last 4 digits of SSN/National ID (for verification)

5. **Bank Account:**

- Add bank account for payouts
- Verify account ownership (may take 1-2 business days)

Step 3: Activate Your Account

1. Complete all required fields
2. Agree to Stripe's Terms of Service
3. Submit for review
4. Wait for approval (usually instant, sometimes 1-2 business days)

Getting API Keys

Stripe provides two sets of API keys:

- **Test keys:** For development and testing (safe to use)
- **Live keys:** For real transactions (handle with care!)

Accessing API Keys

1. **Log in to Stripe Dashboard:** <https://dashboard.stripe.com>
2. **Navigate to API keys:**
 - Click "Developers" in the top right
 - Click "API keys"

Test Keys (Development)

You'll see two test keys:

Publishable key (safe to expose):

```
pk_test_51XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Secret key (keep confidential!):

```
sk_test_51XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Live Keys (Production)

 **Important:** Only use live keys in production, never in development!

Toggle "Test mode" OFF to see live keys:

Publishable key:

```
pk_live_51XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Secret key:

```
sk_live_51XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Adding Keys to Your Application

Development (.env.local):

```
STRIPE_SECRET_KEY=sk_test_51XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_test_51XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Production (Vercel/Railway):

```
STRIPE_SECRET_KEY=sk_live_51XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_live_51XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Webhook Configuration

Webhooks allow Stripe to notify your application when payment events occur (payment succeeded, failed, refunded, etc.).

Why Webhooks Are Important

-  Confirm payment completion
-  Update order status
-  Send confirmation emails
-  Handle asynchronous payment methods
-  Prevent fraud

Local Development Webhook Setup

For testing webhooks locally, use the Stripe CLI.

Step 1: Install Stripe CLI

macOS (Homebrew):

```
brew install stripe/stripe-cli/stripe
```

Windows (Scoop):

```
scoop bucket add stripe https://github.com/stripe/scoop-stripe-cli.git
scoop install stripe
```

Linux:

```
wget https://github.com/stripe/stripe-cli/releases/latest/download/
stripe_X.X.X_linux_x86_64.tar.gz
tar -xvf stripe_X.X.X_linux_x86_64.tar.gz
sudo mv stripe /usr/local/bin/
```

Or download directly: <https://github.com/stripe/stripe-cli/releases>

Step 2: Login to Stripe CLI

```
stripe login
```

This opens a browser to authorize the CLI.

Step 3: Forward Webhooks to Local Server

```
stripe listen --forward-to localhost:3000/api/checkout/webhook
```

You'll see output like:

```
> Ready! Your webhook signing secret is whsec_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Step 4: Copy Webhook Secret

Copy the `whsec_...` secret and add to `.env.local`:

```
STRIPE_WEBHOOK_SECRET=whsec_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Step 5: Restart Development Server

```
# Stop current server (Ctrl+C)
npm run dev
```

Now when you test payments, Stripe CLI forwards events to your local server!

Production Webhook Setup

For your live website, configure webhooks in the Stripe Dashboard.

Step 1: Create Webhook Endpoint

1. **Go to Stripe Dashboard:** <https://dashboard.stripe.com>
2. **Navigate to Webhooks:**
 - Click "Developers" → "Webhooks"
3. **Click "Add endpoint"**

Step 2: Configure Endpoint

1. Endpoint URL:

```
https://yourdomain.com/api/checkout/webhook
```

Replace `yourdomain.com` with your actual domain.

1. Description (optional):

```
Production webhook for order processing
```

2. Events to send:

Select these essential events:

- ☒ `checkout.session.completed`
- ☒ `checkout.session.async_payment_succeeded`

- ☒ `checkout.session.async_payment_failed`
- ☒ `payment_intent.succeeded`
- ☒ `payment_intent.payment_failed`
- ☒ `charge.refunded`
- ☒ `charge.dispute.created`

Or select: “Select all events” (easier, recommended)

1. API version:

- Use latest version (automatically selected)

2. Click “Add endpoint”

Step 3: Get Webhook Signing Secret

1. Click on your newly created webhook endpoint
2. Under “Signing secret”, click “Reveal”
3. Copy the secret (starts with `whsec_...`)

Step 4: Add to Production Environment Variables

In Vercel/Railway, add:

```
STRIPE_WEBHOOK_SECRET=whsec_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Step 5: Test Webhook

1. In Stripe Dashboard, click “Send test webhook”
2. Select `checkout.session.completed`
3. Click “Send test webhook”
4. Check webhook response (should be 200 OK)

Testing Payments

Test Credit Cards

Stripe provides test card numbers for different scenarios:

Successful Payments

Basic successful card:

```
Card number: 4242 4242 4242 4242
Expiry: Any future date (e.g., 12/34)
CVC: Any 3 digits (e.g., 123)
ZIP: Any 5 digits (e.g., 12345)
```

3D Secure authentication required:

```
Card number: 4000 0027 6000 3184
```

Visa (debit):

Card number: 4000 0566 5566 5556

Mastercard:

Card number: 5555 5555 5555 4444

American Express:

Card number: 3782 822463 10005

Payment Failures**Generic decline:**

Card number: 4000 0000 0000 0002

Insufficient funds:

Card number: 4000 0000 0000 9995

Lost card:

Card number: 4000 0000 0000 9987

Stolen card:

Card number: 4000 0000 0000 9979

Expired card:

Card number: 4000 0000 0000 0069

Incorrect CVC:

Card number: 4000 0000 0000 0127

International Cards**UK card:**

Card number: 4000 0082 6000 0000

Australian card:

Card number: 4000 0003 6000 0006

Canadian card:

Card number: 4000 0012 4000 0000

Complete Test Card List

For more test cards: <https://stripe.com/docs/testing#cards>

Testing Checkout Flow

1. Start local server:

```
bash
npm run dev
```

2. Start Stripe CLI webhook forwarding:

```
bash
stripe listen --forward-to localhost:3000/api/checkout/webhook
```

3. Add products to cart**4. Proceed to checkout****5. Use test card:** 4242 4242 4242 4242**6. Complete payment****7. Verify:**

- ☒ Order confirmation page displays
- ☒ Order saved to database
- ☒ Confirmation email sent
- ☒ Webhook event logged in Stripe CLI
- ☒ Payment appears in Stripe Dashboard

Monitoring Test Payments

1. Stripe Dashboard → Payments:

- View all test payments
- See payment details
- Check metadata

2. Stripe Dashboard → Events:

- View all webhook events
- See event payload
- Check delivery status

3. Stripe Dashboard → Logs:

- View API request logs
- Debug errors

Going Live Checklist

Before enabling live payments:

1. Complete Stripe Account Activation

- ☐ Business details completed
- ☐ Bank account added and verified
- ☐ Identity verification completed
- ☐ Tax information submitted (if required)
- ☐ Terms of Service accepted

2. Switch to Live API Keys

- ☐ Update `STRIPE_SECRET_KEY` to `sk_live_...`
- ☐ Update `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY` to `pk_live_...`
- ☐ Update `STRIPE_WEBHOOK_SECRET` to production webhook secret
- ☐ Remove all test keys from production environment

3. Configure Production Webhook

- ☐ Webhook endpoint created at `https://yourdomain.com/api/checkout/webhook`
- ☐ All necessary events selected
- ☐ Webhook signing secret added to environment variables
- ☐ Test webhook sending successful (200 OK response)

4. Test Live Payment

 **Use your own credit card for a real test transaction!**

- ☐ Complete a real purchase with your card
- ☐ Verify order created in database
- ☐ Verify confirmation email received
- ☐ Verify payment appears in Stripe Dashboard
- ☐ Refund the test payment

5. Configure Stripe Settings

Payment methods:

- ☐ Enable desired payment methods (cards, Apple Pay, Google Pay)
- ☐ Set up additional payment methods if needed (SEPA, iDEAL, etc.)

Radar (fraud prevention):

- ☐ Review default Radar rules
- ☐ Enable additional fraud checks
- ☐ Set up custom rules if needed

Email receipts:

- ☐ Customize Stripe receipt emails
- ☐ Add your logo
- ☐ Update support email

Disputes & chargebacks:

- ☐ Set up email notifications for disputes
- ☐ Review dispute policies

6. Legal & Compliance

- ☐ Privacy policy includes payment processing disclosure

- [] Terms clearly state refund policy
- [] PCI compliance maintained (Stripe handles this)
- [] Data handling complies with regulations (GDPR, CCPA)

7. Security Checks

- [] SSL certificate active (HTTPS)
- [] API keys stored securely (never in client-side code)
- [] Webhook signature verification implemented
- [] CORS configured properly
- [] Environment variables not exposed

Currency Configuration

Setting Default Currency

The template is set to Australian Dollars (AUD) by default.

Change Currency in Stripe Checkout

Location: API route creating Checkout Session

File: `pages/api/checkout/create-checkout-session.ts` or similar

```
const session = await stripe.checkout.sessions.create({
  payment_method_types: ['card'],
  line_items: [
    {
      price_data: {
        currency: 'aud', // Change to: 'usd', 'eur', 'gbp', etc.
        product_data: {
          name: product.name,
        },
        unit_amount: product.price * 100, // Amount in cents
      },
      quantity: item.quantity,
    },
  ],
  mode: 'payment',
  success_url: `${process.env.NEXT_PUBLIC_API_URL}/checkout/success?session_id={CHECKOUT_SESSION_ID}`,
  cancel_url: `${process.env.NEXT_PUBLIC_API_URL}/checkout/cancel`,
})
```

Currency Display in Frontend

Update price formatting:

```
// lib/utils.ts
export function formatPrice(amount: number, currency: string = 'AUD') {
  return new Intl.NumberFormat('en-AU', {
    style: 'currency',
    currency: currency, // 'AUD', 'USD', 'EUR', 'GBP'
  }).format(amount)
}
```

Supported Currencies

Stripe supports 135+ currencies. Common ones:

- `usd` - US Dollar
- `aud` - Australian Dollar
- `eur` - Euro
- `gbp` - British Pound
- `cad` - Canadian Dollar
- `jpy` - Japanese Yen
- `nzd` - New Zealand Dollar

Full list: <https://stripe.com/docs/currencies>

Currency Conversion

Note: Stripe charges in the currency you specify. Currency conversion is not handled automatically.

For multi-currency support:

1. Store prices in multiple currencies in database
2. Detect user location
3. Display prices in their currency
4. Process payment in their currency

Common Issues & Solutions

Issue 1: Webhook Signature Verification Failed

Error: No signatures found matching the expected signature for payload

Causes:

- Incorrect `STRIPE_WEBHOOK_SECRET`
- Request body modified before verification
- Using wrong webhook secret (test vs live)

Solutions:

1. Verify you're using the correct webhook secret
2. Ensure request body is raw (not parsed) during verification
3. Check webhook endpoint code:

```
typescript
export const config = {
  api: {
    bodyParser: false, // Required! Must use raw body
  },
}
```

Issue 2: Webhook Not Receiving Events

Error: Webhook never triggers

Causes:

- Webhook URL incorrect
- Firewall blocking requests

- Server not running
- Wrong events selected

Solutions:

1. Verify webhook URL in Stripe Dashboard
2. Test webhook in Stripe Dashboard ("Send test webhook")
3. Check server logs for incoming requests
4. Ensure server is publicly accessible (not localhost in production)

Issue 3: Payment Succeeds But Order Not Created

Error: Payment completed but order missing from database

Causes:

- Database connection error in webhook handler
- Error in order creation logic
- Webhook handler crashing before completing

Solutions:

1. Check webhook logs in Stripe Dashboard
2. Add extensive error logging:

```
typescript
try {
  // Order creation logic
} catch (error) {
  console.error('Order creation failed:', error)
  // Log to external service (Sentry, etc.)
}
```

3. Implement retry logic for failed webhooks
4. Check database connection in webhook handler

Issue 4: Test Payments Work, Live Payments Don't

Error: Switching to live mode breaks payments

Causes:

- Still using test API keys
- Live webhook not configured
- Account not fully activated

Solutions:

1. Double-check all API keys are live keys
2. Verify webhook endpoint exists for live mode
3. Complete account activation in Stripe
4. Check for any account restrictions

Issue 5: Metadata Not Appearing in Stripe

Error: Custom metadata missing from Stripe payment

Causes:

- Metadata not being sent in API call
- Exceeding metadata limits (50 keys, 500 chars per value)

Solutions:

1. Add metadata to checkout session:

```
typescript
const session = await stripe.checkout.sessions.create({
  metadata: {
    order_id: '12345',
    customer_email: 'customer@example.com',
    cart_items: JSON.stringify(items).slice(0, 500),
  },
})
```

Issue 6: Customers Not Receiving Stripe Receipts

Error: Stripe email receipts not sent

Causes:

- Customer email not provided
- Stripe receipts disabled
- Email going to spam

Solutions:

1. Ensure customer email is passed:

```
typescript
customer_email: 'customer@example.com',
```

2. Enable receipts in Stripe Dashboard
3. Customize email to improve deliverability

Issue 7: Duplicate Charges

Error: Customer charged multiple times

Causes:

- Multiple form submissions
- Idempotency not implemented
- Retry logic triggering duplicate charges

Solutions:

1. Implement idempotency keys:

```
typescript
const session = await stripe.checkout.sessions.create({
  // ... other params
}, {
  idempotencyKey: `order_${orderId}_${timestamp}`,
})
```

2. Disable submit button after first click
3. Add loading state during checkout

Payment Flow Overview

Understanding the complete payment flow:

1. Customer Initiates Checkout

Cart → Checkout Button Clicked

2. Create Checkout Session

```
// Frontend: app/checkout/page.tsx
const response = await fetch('/api/checkout/create-checkout-session', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ items: cartItems }),
})

const { sessionId } = await response.json()

// Redirect to Stripe Checkout
const stripe = await loadStripe(process.env.NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY)
stripe.redirectToCheckout({ sessionId })
```

3. Backend Creates Session

```
// Backend: pages/api/checkout/create-checkout-session.ts
const session = await stripe.checkout.sessions.create({
  payment_method_types: ['card'],
  line_items: [...],
  mode: 'payment',
  success_url: `${domain}/checkout/success?session_id={CHECKOUT_SESSION_ID}`,
  cancel_url: `${domain}/checkout/cancel`,
  customer_email: email,
  metadata: { order_id: orderId },
})

return res.json({ sessionId: session.id })
```

4. Customer Completes Payment on Stripe

Customer → Enters card details → Submits → Stripe processes

5. Stripe Sends Webhook Event

Stripe → POST → <https://yourdomain.com/api/checkout/webhook>

6. Webhook Handler Processes Event

```
// Backend: pages/api/checkout/webhook.ts
const event = stripe.webhooks.constructEvent(body, signature, webhookSecret)

switch (event.type) {
  case 'checkout.session.completed':
    const session = event.data.object

    // Create order in database
    await createOrder(session)

    // Send confirmation email
    await sendOrderConfirmation(session)

    break
}

return res.json({ received: true })
```

7. Customer Redirected to Success Page

Stripe → Redirects → https://yourdomain.com/checkout/success?session_id=...

8. Success Page Displays

```
// Frontend: app/checkout/success/page.tsx
const sessionId = searchParams.get('session_id')

// Fetch order details
const order = await getOrderById(sessionId)

return <OrderConfirmation order={order} />
```

Advanced Features

Saving Cards for Future Use

Enable customer card saving:

```
const session = await stripe.checkout.sessions.create({
  mode: 'payment',
  payment_intent_data: {
    setup_future_usage: 'on_session', // Save card for future
  },
  // ... other params
})
```

Subscriptions

For recurring payments (subscriptions):

```
const session = await stripe.checkout.sessions.create({
  mode: 'subscription', // Change from 'payment' to 'subscription'
  line_items: [
    {
      price: 'price_XXXXXXXXXXXXXXXXXXXX', // Stripe Price ID
      quantity: 1,
    },
  ],
  // ... other params
})
```

Coupons & Discounts

Apply discount codes:

```
const session = await stripe.checkout.sessions.create({
  discounts: [{
    coupon: 'SUMMER2024', // Coupon code from Stripe Dashboard
  }],
  // ... other params
})
```

Shipping Rates

Add shipping options:

```
const session = await stripe.checkout.sessions.create({
  shipping_address_collection: {
    allowed_countries: ['AU', 'US', 'GB'],
  },
  shipping_options: [
    {
      shipping_rate_data: {
        type: 'fixed_amount',
        fixed_amount: {
          amount: 500, // $5.00
          currency: 'aud',
        },
      },
      display_name: 'Standard Shipping',
      delivery_estimate: {
        minimum: { unit: 'business_day', value: 5 },
        maximum: { unit: 'business_day', value: 7 },
      },
    },
  ],
  // ... other params
})
```

Resources

Official Documentation

- **Stripe Documentation:** <https://stripe.com/docs>
- **Stripe API Reference:** <https://stripe.com/docs/api>

- **Checkout Documentation:** <https://stripe.com/docs/payments/checkout>
- **Webhooks Guide:** <https://stripe.com/docs/webhooks>
- **Testing Guide:** <https://stripe.com/docs/testing>

Support

- **Stripe Support:** <https://support.stripe.com>
- **Stripe Community:** <https://stripe.com/community>
- **Stack Overflow:** <https://stackoverflow.com/questions/tagged/stripe-payments>

Tools

- **Stripe CLI:** <https://stripe.com/docs/stripe-cli>
- **Stripe Shell:** <https://stripe.com/docs/stripe-shell>
- **API Explorer:** <https://stripe.com/docs/api#intro>

Congratulations! 🎉 Your Stripe payment system is now fully configured!

Next Steps:

- Test thoroughly with test cards
- Complete “Going Live” checklist
- Monitor your first real payments
- Set up email notifications for payments and disputes

Last Updated: November 2024